



Da MCP tools a **Distributed Intelligence**

Come DDD, MCP, A2A, RAG, Wiki e observability cambiano il modo in cui progettiamo sistemi AI-native

Ferdinando Santacroce · Alberto Acerbis
InnovAI 2026 · github.com/BrewUp/BrewUpErp

La tesi

**Progettare un sistema AI-native
non significa aggiungere agenti a un'applicazione.**

**Significa progettare
come l'intelligenza fluisce
nell'organizzazione.**

- Dove vive la conoscenza?
- Chi ha l'autorità di rispondere?
- Chi coordina quando servono più contesti?
- Come sappiamo che ha funzionato davvero?

Il laboratorio: BrewUp ERP

Un ERP didattico costruito con i vincoli del mondo reale: confini espliciti, niente scorciatoie.

Modular monolith

Bounded context isolati
nello stesso deployable,
nessuna dipendenza diretta
tra moduli

DDD + CQRS

Aggregati, eventi di
dominio, saghe e read
model separati dalle
scritture

Event-driven

Comunicazione solo tramite
messaggi espliciti, mai
accesso condiviso ai dati

Stack .NET 10

Muflone, KurrentDB
(EventStore), RabbitMQ,
MongoDB

Quattro contesti: MasterData · Sales · Warehouse · Knowledge
tag `demo/00-ddd-base`

Quattro contesti, quattro linguaggi

MasterData

Prodotti, listini, anagrafiche.
È la fonte di verità del catalogo.

Sales

Ordini di vendita, prezzi,
sconti, valutazioni
commerciali.

Warehouse

Giacenze, movimenti,
disponibilità e impatti sulla
produzione.

Knowledge

Contratti, procedure,
documenti: la conoscenza
che non sta in una tabella.

**Nessun contesto "sa tutto".
È il vincolo, non il problema.**



Creare un agente è facile.
Progettarlo no.

Le demo mostrano agenti che funzionano. La differenza non è nella demo: è in **dove vive la conoscenza, chi decide e chi coordina.**



La mappa del talk

MCP → confini → conoscenza → agenti → coordinazione → osservabilità

Esperimento 1 — dare all'AI accesso all'ERP

Un solo **MCP Server** che espone un catalogo di tool: le capacità dell'ERP rese scopribili.

MCP (Model Context Protocol) è la superficie scopribile: l'agente vede **cosa può fare**, senza conoscere come è fatto l'ERP dentro.

- Un server, tanti tool, tante responsabilità
- Si parte dal caso più semplice:
rispondere a domande sui dati

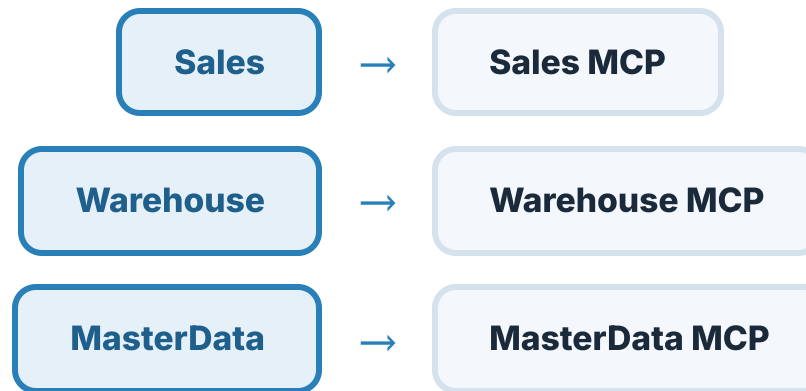
tag `demo/01-mcp-monolith` — "Add Chat and MCP" · `.../AI/BrewUp.AI.McpServer/Tools/BrewUpMcpTools.cs:9`

Problema: il Tool Catalog diventa un **monolite**

- 30+ tool, 4 domini, una sola superficie
- Il confine del dominio si perde nel catalogo
- Ownership e manutenzione non sono più chiari
- Un cambiamento in Sales tocca il server che espone tutto

**Abbiamo ricreato in AI il monolite
che il DDD ci aveva fatto evitare.**

Decisione: un MCP Server per Bounded Context



Il catalogo torna a coincidere con i confini del **dominio**.

tag `demo/02-mcp-per-bc` – monolite rimosso, un MCP per contesto · `.../MasterData/BrewUp.MasterData.McpServer/Tools/MasterDataTools.cs:7`

Ogni contesto espone il **proprio** catalogo

Sales MCP

Ordini, prezzi, valutazioni commerciali.
Paginazione per efficienza.

Warehouse MCP

Giacenze, disponibilità, impatti sulla
produzione.

MasterData MCP

Prodotti e listini, la fonte di verità del
catalogo.

Un tool appartiene a un contesto, e a uno solo.

Esperimento 2 — chi **coordina** i contesti?



Un coordinatore che **delega**, non un super-agente che sa tutto.

tag `demo/03-orchestrator` — "Introduce agent-based coordination for what-if analysis" · `.../Mother.Facade/Agents/MotherCoordinator.cs:63`

Il what-if passo per passo



**Nessun agente risponde da solo:
il risultato è una composizione.**

Il dominio non vive tutto nel database

Contratti, procedure, PDF, wiki operative: conoscenza che non sta in una tabella.

Knowledge Context

Un bounded context
dedicato alla conoscenza

Ingestion

PDF e Markdown,
estrazione, chunking

Embedding

Azure OpenAI per i vettori

Retrieval

Vector store: InMemory,
SQL Server, Azure AI
Search

tag `demo/04-rag` – "RAG completed" · tool `search_knowledge_base` · `.../Knowledge.McpServer/Tools/KnowledgeTools.cs:10`

RAG: dalla fonte alla risposta



- L'ingestion trasforma documenti in vettori ricercabili
- Il retrieval restituisce i chunk più simili alla domanda
- Tutto dentro i confini del Knowledge Context

Retrieval **non** significa conoscenza

Un chunk recuperato non è una risposta **giustificata**.

- Il retrieval risponde a "cosa è simile"
- Non risponde a "qual è la fonte"
- E non dice se il risultato è ancora valido

**Serve conoscenza derivata, con
provenance.**

Un tool non è un agente

MCP

"What can you do?" — espone capacità, non si assume obiettivi

A2A

"Can you take responsibility?" — assume un obiettivo e lo porta a termine

Il **Knowledge Agent**: dal tool al collaboratore.

tag `demo/05-a2a` — A2A senza framework · `.../BrewUp.Shared/Agents/AgentCard.cs:3` · `.../Knowledge.Facade/Agents/KnowledgeAgentCardProvider.cs:7`

A2A: agenti che si parlano

- Ogni agente pubblica una AgentCard: identità e capacità
- Un agente scopre un altro agente e gli delega un obiettivo
- Il confine resta esplicito: è un handoff, non una chiamata di funzione

Il Knowledge Agent espone se stesso via A2A e a sua volta usa MCP per parlare col Knowledge Context.



Il sistema cresce

Orchestrazione e trasporto: **Aspire** + A2A su HTTP + Service Bus.

Il protocollo stesso si evolve: l'aggiornamento a MCP 2.0 arriva dopo, insieme alla parte finale del sistema. Nell'ordine narrativo resta un aggiornamento, non un nuovo passo.

```
demo/05-a2a → demo/06-aspire → demo/08-llm-wiki ·  
.../Mother.Facade/Mcp/McpToolsProvider.cs:52 ·  
.../BrewUp.Shared/Agents/McpToolClient.cs:25
```

Ha funzionato non è abbastanza

Technical observability

Latenza, span, errori. Sappiamo **che** è successo.

Semantic observability

AgentRun, workflow, capability, handoff, outcome. Sappiamo **cosa** e **perché**.

Apriamo il trace.

tag `demo/07-observability` – telemetria semantica nel sistema finale • `.../Mother.Facade/Telemetry/MotherTelemetry.cs:28`

Cosa tracciamo

- AgentRun: l'esecuzione di un agente dall'inizio alla fine
- Workflow: la sequenza di passi che compongono un what-if
- Capability: quale capacità è stata invocata e con quale esito
- Handoff: il passaggio di responsabilità tra agenti
- Outcome: il risultato finale, valutabile

Osservabilità semantica = capire, non solo misurare.

LLM Wiki: conoscenza derivata con **provenance**

Documenti → Wiki generata → risposta con **riferimenti alla fonte**.

search_knowledge_base

query_wiki

get_wiki_page

get_wiki_page_evidence

tag `demo/08-llm-wiki` – il finale · KnowledgeTools · `.../Knowledge.McpServer/Tools/KnowledgeTools.cs:35,36`

Perché una Wiki e non solo RAG

- La conoscenza è derivata, non solo recuperata
- Ogni pagina mantiene i riferimenti alle fonti (provenance)
- Si aggiorna in modo controllato, non a ogni query
- L'agente può citare la fonte, non solo rispondere

Una risposta senza fonte è un'opinione.

Evaluation

- Non basta che il sistema risponda: deve rispondere **bene**.
- KnowledgeRetrievalEvaluator: misura la qualità del retrieval
 - Valutazione continua, non una demo una tantum
 - La qualità della conoscenza è un requisito, non un extra

L'osservabilità dice cosa ha fatto il sistema. L'evaluation dice se era sufficiente.

```
tag demo/09-evaluation · .../Knowledge.Facade/Evaluation/KnowledgeRetrievalEvaluator.cs:10 ·  
.../Mother.Facade/Agents/WhatIfWorkflowEvaluator.cs:45
```

Il sistema finale: intelligence flow



- **Bounded Contexts** con MCP Server separati
- **KnowledgeTools** con provenance
- **Knowledge Agent**: Mother → A2A → Agent → MCP → Knowledge



Show me the system

Demo end-to-end: **"What if we sell 500 bottles of IPA?"**





Grazie

alberto.acerbis@intre.it · ferdinando.santacroce@gmail.com

<https://github.com/BrewUp/BrewUpErp>